

CITS3002 Group Project

Student ID	Name
24494921	Benjamin Passaportis
24527669	Tinashe Nemacha

Overview

This project implements a simplified network topology simulator in python to demonstrate how data is transmitted between two hosts through a router using the Data Link, Network, and Transport layers. Key features include encapsulation, MAC and IP addressing, routing, frame forwarding, checksum verification, and reliable data transferral using the rdt2.2 alternating bit protocol. Data is segmented into 500 byte segments, encapsulated into IP-like packets and Ethernet-like frames, then forwarded across the network while outputting detailed logs at every layer to get an idea of end-to-end network communication.

Usage

```
python main.py <message_size>
```

Where `<message_size>` is the size of the segment payload in bytes.

Directory Structure

```
/
├─ config.py
├─ devices.py
├─ main.py
├─ protocol.py
└─ README.md
```

- `config.py` : stores hard-coded constants and helper-functions as per the project specification.
- `devices.py` : stores the classes that define the different network devices.
 - `Topology` : stores all the devices in the network enabling inter-device communication.
 - `Device` : superclass for the `Host` and `Router` classes. Simply exists to simplify logging.
 - `Host` : defines a host on the network, as well as all its constituent layers.
 - `Router` : defines a router on the network, including its layers and different interfaces.
- `main.py` : the entry point for the network simulator. Takes the size of the message as a command-line argument.
- `protocol.py` : defines the structure and functionality of the different layers.
 - `Frame` : defines the frame header and payload structure. Utilised by the data link layer.
 - `Packet` : defines the packet header and payload structure. Utilised by the network layer.
 - `Segment` : defines the segment header and payload structure. Utilised by the transport layer.
 - `Layer` : superclass for the remaining layers to simplify logging.
 - `DataLinkLayer` : the structure and functionality of layer 2 for both the hosts and the router.
 - `NetworkLayer` : the structure and functionality of layer 3 for both the hosts and the router.
 - `TransportLayer` : the structure and functionality of layer 4 for both the hosts and the router.
- `README.md` : a more succinct version of this document.

Functionality

- Data link layer
 - Framing and encapsulation
 - MAC addressing

- Switching/forwarding
- Frame delivery
- Network layer
 - Encapsulation
 - IP addressing
 - Routing/forwarding
 - TTL handing
 - Packet delivery
- Transport layer
 - Segmentation at 500 byte payload size
 - Encapsulation
 - Port-based communication
 - *Note: this is largely arbitrary given that there are no actual applications running in the application layer*
 - Length handing
 - Checksum and error detection using rdt2.2 protocol
 - *Note: whilst this is fully implemented, the actual specification of the problem prevents it from being functionally used at any point. Certain choices were made where the specification is ambiguous. Where these occur, they are accompanied with comments in the code*

Data Link Layer Methods

encapsulate()

- Packs the packet into a frame with the following header information:
 - Destination MAC
 - Source MAC
 - Type
 - Payload (packet)

decapsulate()

- Fetches the payload from a frame

send()

- Gets the device MAC address
- Receives the packet from the network layer
- Uses the ARP table to find the destination MAC address
- Uses the network topology to find the address and interface of the destination
- Sends the frame to the data link layer of the receiving device

receive()

- Parses and decapsulates the received frame
- Updates the current interface if on a router
- Sends the packet to the network layer

Network Layer Methods

encapsulate()

- Packs the segment into a packet with the following header information:
 - Source IP
 - Destination IP
 - TTL
 - Protocol
 - Total length
 - Payload (segment)

decapsulate()

- Fetches the payload from a packet

send()

- Receives a segment from the transport layer
- Determines the next-hop IP from the routing table

- If in a router, determines the required outgoing interface for the next-hop
- Encapsulates the packet and sends to the data link layer

receive()

- Receives a packet from the data link layer
- Router
 - Discards the packet if TTL = 0
 - Decrements the TTL
 - Determines the next-hop IP from the routing table
 - Selects the appropriate outgoing interface
 - Encapsulates and sends to the data link layer
- Host
 - Decapsulates the packet into a segment
 - Sends the segment to the transport layer

Transport Layer Methods

encapsulate()

- Packs the data into a segment with the following header information:
 - Source port
 - Destination port
 - Length
 - Checksum
 - Type (Data/ACK)
 - Sequence number
 - Payload

decapsulate()

- Fetches the payload from the segment and decodes it back into plaintext

send()

- Recieves data from the application layer (user-input)
- Segments the data into 500 byte chunks
- Sends each encapsulated chunk to the network layer and blocks until positive ACK is recieved
- Done using rdt2.2 with alternating sequence numbers

receive()

- Recieves a segment from the network layer and parses it
- Verifies the checksum and discards the segment if incorrect
- Data segments:
 - Verifies the sequence number and request resend if incorrect
 - Constructs an ACK to send to the network layer
- ACK:
 - Verifies sequence number and resends last chunk if incorrect (resend requested)
 - Sends an ACK to the network layer and unblocks

checksum()

- Creates a two-byte checksum using modulo 256

verify_checksum()

- Verifies that the data-calculated and recieved checksums match